[code_analysis] Semantic check - Type Checking and Constant Folding

Created By	Hyung-Gyu Ryoo
Email	hgryoo@cubrid.com
Tags	코드분석
Team	개발2팀
Version	1.0

Overview
Type Evaluation for Expression - pt_eval_expr_type()
- 룰 기반의 type evaluation
- Expression definition에 따라 type evaluation
- Collation을 가지는 경우 collation compatible 확인
- Host variable에 대해 late binding하는 operator에 대해 타입 변환
- 인수의 타입에 따라 Expression의 타입이 자명해지는 경우 타입 설정
- 타입이 정해진 host variable에 대해 인수 타입 설정
- Recursive Expression

Type Evaluation for Function

Overview

이 단계에서는 파싱 트리를 모두 순회하며 Expression (PT_EXPR)과 Function (PT_FUNCTION)에 대해 타입을 확인하고, 상수인 경우에 하나의 VALUE로 만드는 것에 대해 설명한다. 이를 각각 Type checking과 Constant folding이라고 한다. pt_semantic_type() 함수에서 이 두가지 동작을 수행하는데 Query Processing 메인 루틴에서 전체 파싱 트리에 대해 최소 한번은 호출되고, 서브트리의 변경이 있어 타입에 대해 재확인이 필요할 때마다 해당 서브트리에 대해 이 함수를 호출한다.

```
PT_NODE *
pt_semantic_type (PARSER_CONTEXT * parser, PT_NODE * tree, SEMANTIC_CHK_INFO * sc_info_ptr)
{
  SEMANTIC_CHK_INFO sc_info = { tree, NULL, 0, 0, 0, false, false };

  if (pt_has_error (parser))
    {
      return NULL;
    }
  if (sc_info_ptr == NULL)
    {
      sc_info_ptr = &sc_info;
    }
  /* do type checking */
  tree = parser_walk_tree (parser, tree, pt_eval_type_pre, sc_info_ptr, pt_eval_type, sc_info_ptr);
  /* do constant folding */
  tree = parser_walk_tree (parser, tree, pt_fold_constants_pre, NULL, pt_fold_constants_post, sc_info_ptr);
  if (pt_has_error (parser))
    {
      tree = NULL;
    }

  return tree;
}
```

pt_semantic_type()가 호출하는 함수들에서 수행하는 것을 간단히 설명하고, 자세한 설명이 필요한 경우 다음 장에서 다룬다.

- **pt_eval_type_pre()**
  노드에 진입할 때 호출된다. 각각의 노드 타입마다 다음을 수행한다.
  - **PT_SPEC:** derived_table이고 query인 경우 (SELECT, UNION, DIFFERENCE, INTERSECTION), 만약 outer join이 존재하면 has_outer_spec flag를 표시둔다. 주석에서는 outer join spec이 folding되지 않도록 표시한다고 설명되어 있다.
  - **PT_SORT_SPEC:** sort spec expression이 query인 경우 is_sort_spec flag를 표시한다.
  - **PT_SELECT를 포함한 query 노드들:** LIMIT를 적절한 Numbering Expression으로 변환한다. 이 내용은 앞의 문서를 참고한다. (Semantic check - Checking Semantic of Particular Statement#Search Condition 관련)
  - **PT_EXPR:** Recursive Expression에 대해 타입 검사와 변환을 수행한다. 이 내용은 별도의 장에서 다룬다.
- **pt_eval_type()**
  - **PT_EXPR:** pt_eval_expr_type () 함수에서 Expression의 인수의 타입과 결과 타입을 확인하고 지정한다.
  - **PT_FUNCTION:** pt_eval_function_type () 함수에서 Function의 인수의 타입과 결과 타입을 확인하고 지정한다.
  - **PT_CREATE_INDEX, PT_DELETE, PT_UPDATE:** 이 노드가 가진 search condition들에 대해 필요 없는 부분을 제거한다. 이 내용은 앞의 문서를 참고한다. (Semantic check - Checking Semantic of Particular Statement#Search Condition 관련)
  - **PT_MERGE, PT_SELECT:** search condition에 대해 필요 없는 부분을 제거하고, 하위 노드로부터 타입을 지정한다. (각각 values 또는 select list)
- **pt_fold_constants_pre()**
  - 단순히 benchmark 함수가 있는 경우 benchmark 대상이 CAS에서 constant folding 되지 않고 executor에서 실행되도록 하기 위해 더 이상 순회를 진행하지 않는다. (PT_LIST_WALK 이용, 자식 노드로 진행하지 않고 node→next까지만 진행한다)
- **pt_fold_constants_post()**
  - Expression(PT_EXPR)과 Function(PT_FUNCTION)에 대해 모두 상수 인수를 가지고 있으면 이 단계에서 실행하여 하나의 PT_VALUE 노드로 만든다.

Constant Folding은 로직이 복잡하지 않기 때문에 이 장에서 설명한 것으로 충분하다. 다음 장에서 Type evaluation에 대해 자세히 설명한다.

Type Evaluation for Expression - pt_eval_expr_type()

PT_EXPR은 인수로 최대 3개까지, arg1, arg2, arg3를 가질 수 있다. 이 로직 내에서 만약 Host Variable이면 arg1_hv, arg2_hv, arg3_hv 변수로 각각 관리한다.

이 단계의 목표는 PT_EXPR의 type_enum과 data_type 노드를 operator 타입과 인수로부터 정하는 것이다. 복잡하게 구현되어 있지만 이 함수의 전체 동작을 요약하면 다음과 같다.

- operator에 따라 룰 기반으로 type evaluation
- expression definition에 따라 type evaluation
- collation을 가지는 경우 collation compatible 확인
- Host variable에 대해 late binding하는 operator에 대해 타입 변환
- 인수의 타입에 따라 Expression의 타입이 자명해지는 경우 타입 설정
- 타입이 정해진 host variable에 대해 인수 타입 설정

룰 기반의 type evaluation

다음 operator에 대해 룰 기반으로 타입을 정한다.

- PT_PLUS
- PT_MINUS
- PT_BETWEEN_AND, PT_BETWEEN_GE_LE, PT_BETWEEN_GE_LT, PT_BETWEEN_GT_LE, PT_BETWEEN_GE_LT
- PT_BETWEEN, PT_NOT_BETWEEN
- PT_LIKE, PT_NOT_LIKE
- PT_LIKE_LOWER_BOUND, PT_LIKE_UPPER_BOUND
- PT_IS_IN, PT_IS_NOT_IN
- PT_TO_CHAR
- PT_FROM_TZ, PT_NEW_TIME

Expression definition에 따라 type evaluation

pt_apply_expressions_definition() 함수에서 수행한다.

Collation을 가지는 경우 collation compatible 확인

pt_check_expr_collation ()

Host variable에 대해 late binding하는 operator에 대해 타입 변환

인수가 host variable인 경우 어떤 타입의 값일 지 모르기 때문에 PT_EXPR의 타입은 PT_TYPE_MAYBE가 된다. (물론 operator에 따라 결과 타입이 자명한 경우도 있다) 이 경우에 정확한 타입은 정할 수 없지만 만들어져야하는 domain을 정해두는데, 이것을expected_domain이라고 한다.

operator에 따라 host variable 인수에 대해서 late binding을 해야하는 것들이 있는데 pt_is_op_hv_late_bind() 함수를 참고한다. 이러한 operator의 경우에 expected domain에 대해 cast (PT_CAST 생성) 해두고 XASL Generation 단계에서 DB_TYPE_VARIABLE domain으로 만들어주기 위해 expected_domain을 NULL로 만들어둔다.

인수의 타입에 따라 Expression의 타입이 자명해지는 경우 타입 설정

- PT_BETWEEN, PT_NOT_BETWEEN
- PT_LIKE, PT_NOT_LIKE
- PT_RAND, PT_RANDOM, PT_DRAND, PT_DRANDOM
- PT_EXTRACT
- PT_YEAR, PT_MONTH, PT_DAY
- PT_HOUR, PT_MINUTE, PT_SECOND
- PT_MILLISECOND
- PT_COALESCE
- PT_FROM_TZ

타입이 정해진 host variable에 대해 인수 타입 설정

host variable에 대해 타입이 정해졌다는 것은 arg1_hv, arg2_hv로 가리키고 있던 노드의 타입이 PT_TYPE_MAYBE가 아닌 타입으로 만들어졌다는 것이다. Expression의 operator에 따라 host variable(?) 이라도 그 인수의 타입이 자명하여 정해질 수 있는 경우이다. 이 경우에 Expected domain을 설정해둔다. 물론 이 단계에서 타입이 정해지더라도 Executor 단계에서 전혀 compatible하지 않은 타입이 입력된다면 Executor 단계에서 에러가 발생할 것이다.

Recursive Expression

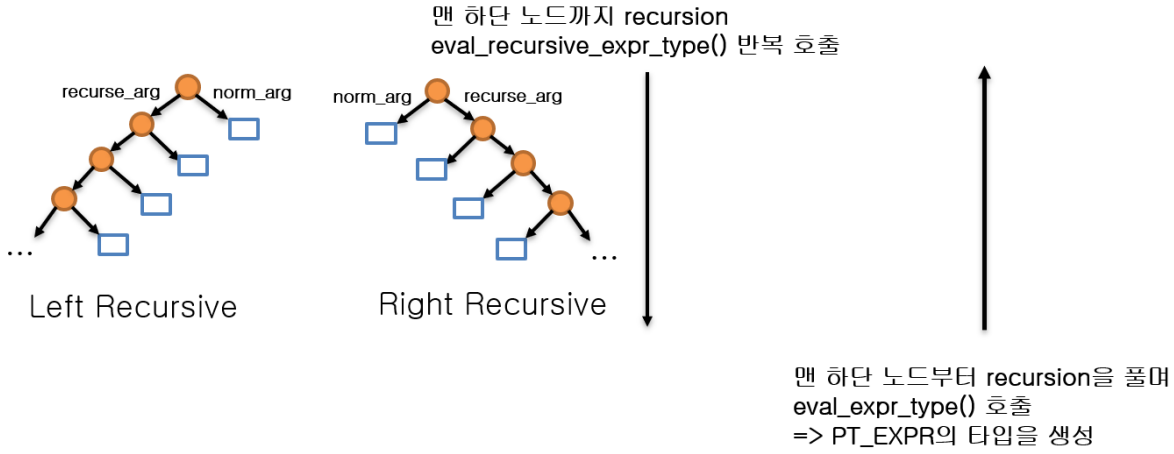
앞서 설명한 단계들은 각 PT_EXPR 노드를 만났을 때 호출된다. 이 때 파싱 트리의 하위에 위치하는 노드부터 타입이 만들어진다. 그러나 몇가지 Expression의 operator 중에 재귀적으로 동일한 operator를 사용하여 하나의 Expression처럼 동작하는 것이 있다.

아래 그림을 참고하면 트리의 왼쪽으로 동일한 operator를 가지는 노드가 반복되는 Left Recursive Expression, 그리고 트리의 오른쪽으로 반복되는 Right Recursive Expression이 있다.

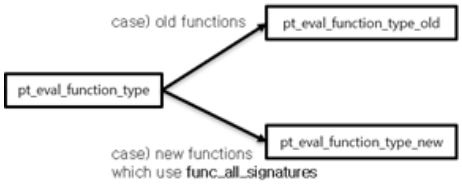
이러한 노드들은 하나의 Expression처럼 취급하여 한꺼번에 타입이 정해져야한다. 따라서 파싱 트리의 상단부터 진입하는 pt_eval_type_pre () 함수에서 이러한 Recursive Expression을 다루는 로직이 구현되어 있다. 그 동작은 아래와 같다.

Left
- GREATEST, LEAST, COALESCE

Right
- CASE, DECODE



Type Evaluation for Function

Function의 경우에는 최대 3개의 인수를 가지는 Expression과 다르게 그 이상의 인수도 가질 수 있다. 그러나 Function (PT_FUNCTION)에 대한 type evaluation도 PT_EXPR과 거의 동일하다. 따라서 이 장에서는 Function의 경우 Expression과 맥락이 거의 비슷하기 때문에 자세한 동작을 설명하기 보다는 현재 어떻게 구현되어 있는지를 설명한다.

Function에 대해 type evaluation은 두가지 로직으로 분기된다. 큐브리드에서 C++ 문법을 도입하면서, SQL 함수를 evaluation하는 여러가지 단계를 하나의 C++ 클래스내에서 동작하도록 구현하였다. func_type 네임스페이스의 Node 클래스를 도입하고 이 클래스 내에서 타입 확인과 변환을 수행한다. 이 때 function signature라는 구조로 각 인수와 반환 타입을 미리 만들어두고 이것과 매칭하여 인수와 반환 타입을 지정한다. 자세한 내용은 func_type.cpp 파일을 참고한다.